Module 4 – Introduction to DBMS

1. Introduction to SQL:

Q.1. What is SQL, and why is it essential in database management?

Ans: **SQL (Structured Query Language)** is a standard language used to **communicate with databases**.
It allows us to **store, retrieve, update, delete, and manage data** in relational databases.

SQL is essential because:

1. **Data Access**
   It helps us easily read and write data from databases.
2. **Data Control**
   We can control who can access data using permissions.
3. **Data Integrity**
   SQL maintains accuracy using constraints like primary key, foreign key, etc.
4. **Fast Querying**
   It retrieves large amounts of data quickly using optimized queries.
5. **Works with All Major Databases**
   Almost every relational database supports SQL.
6. **Backend Requirement**
   In real applications (web or mobile), SQL connects frontend with stored data.

Q.2. Explain the difference between DBMS and RDBMS.

Ans: **Difference Between DBMS and RDBMS**

Feature	DBMS	RDBMS
Full Form	Database Management System	Relational Database Management System
Data Storage	Stores data in files	Stores data in tables (rows & columns)
Relationship	No relationship between tables	Tables are related using primary & foreign keys
Normalization	Not supported	Supported

Feature	DBMS	RDBMS
Data Integrity	Low	High
Security	Less secure	More secure
Users	Single user system	Multi-user system
Size	Suitable for small applications	Suitable for large applications
Examples	FoxPro, dBase	MySQL, Oracle, SQL Server

Q.3. Describe the role of SQL in managing relational databases.

Ans: Role of SQL in Managing Relational Databases

SQL (Structured Query Language) plays a central role in managing relational databases. It is the standard language used to define, manipulate, and control data stored in relational database systems.

1. Data Definition

SQL is used to create and modify the structure of database objects such as tables, views, indexes, and constraints. Commands like CREATE, ALTER, and DROP are part of this function.

2. Data Manipulation

SQL allows users to insert, update, delete, and retrieve data from database tables using commands such as INSERT, UPDATE, DELETE, and SELECT.

3. Data Querying

SQL enables efficient retrieval of data using conditions, sorting, grouping, and joins. It helps generate reports and extract meaningful information from large datasets.

4. Data Integrity

SQL enforces integrity constraints such as Primary Key, Foreign Key, Unique, and Not Null to ensure accuracy and consistency of data.

5. Access Control and Security

SQL provides mechanisms to manage user permissions and control access to database objects through commands like GRANT and REVOKE.

Conclusion

SQL is essential in relational database management as it provides a standardized way to create database structures, manage data, maintain integrity, and ensure security.

Q.4. What are the key features of SQL?

Ans: **Key Features of SQL**

SQL (Structured Query Language) is a standard language used to manage and manipulate relational databases. Its key features are as follows:

1. Data Definition

SQL allows users to define and modify database structures such as tables, views, and indexes using commands like CREATE, ALTER, and DROP.

2. Data Manipulation

SQL supports inserting, updating, deleting, and retrieving data from tables through commands such as INSERT, UPDATE, DELETE, and SELECT.

3. Data Querying

SQL provides powerful querying capabilities, including filtering, sorting, grouping, and joining multiple tables to extract meaningful information.

4. Data Integrity

SQL maintains data accuracy and consistency using constraints like Primary Key, Foreign Key, Unique, and Not Null.

5. Security and Access Control

SQL offers security features to manage user access through GRANT and REVOKE, ensuring that only authorized users can access or modify data.

6. Transaction Control

SQL supports transaction management using commands like COMMIT, ROLLBACK, and SAVEPOINT to ensure reliable data operations.

7. Portability

SQL is supported by most relational database systems, making it portable across platforms such as MySQL, Oracle, SQL Server, and PostgreSQL.

8. High Performance

SQL efficiently handles large volumes of data and provides fast data retrieval through optimized query execution.

Conclusion

SQL provides a complete set of features for defining database structures, managing data, maintaining integrity, ensuring security, and controlling transactions, making it an essential tool in relational database management systems.

2. SQL Syntax:

Q.1. What are the basic components of SQL syntax?

Ans: **Basic Components of SQL Syntax**

SQL syntax consists of several fundamental components that are used to write queries and manage relational databases. These components define how data is created, accessed, modified, and controlled.

1. Keywords

Keywords are reserved words in SQL that perform specific operations.

Examples:
SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, WHERE, FROM

They define the action to be performed.

2. Identifiers

Identifiers are names given to database objects such as tables, columns, views, and indexes.

Examples:

- Table name
- Column name
- Database name

They are user-defined.

3. Clauses

Clauses specify conditions or control how data is retrieved.

Common clauses include:

- FROM – specifies the table
- WHERE – applies conditions
- GROUP BY – groups rows
- HAVING – filters grouped data
- ORDER BY – sorts results

4. Operators

Operators are used to perform comparisons or calculations.

Types of operators:

- Arithmetic: +, -, *, /
- Comparison: =, >, <, >=, <=, !=
- Logical: AND, OR, NOT

5. Expressions

Expressions combine columns, operators, and values to produce a result.

Example:
salary * 12

6. Functions

Functions perform operations on data and return a value.

Examples:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

7. Literals

Literals are fixed values used directly in queries.

Examples:

- Numbers: 1000
- Strings: 'John'
- Dates: '2026-02-24'

Conclusion

The basic components of SQL syntax include keywords, identifiers, clauses, operators, expressions, functions, and literals. Together, they form structured queries that allow effective interaction with relational databases.

Q.2. Write the general structure of an SQL SELECT statement.

Ans:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Q.3. Explain the role of clauses in SQL statements.

Ans: Role of Clauses in SQL Statements

Clauses in SQL define how data is selected, filtered, grouped, and ordered within a query. Each clause performs a specific function and helps control the output of SQL statements.

The main SQL clauses and their roles are:

1. SELECT Clause

Specifies the columns or expressions to be retrieved from the database.

2. FROM Clause

Indicates the table or tables from which data is fetched.

3. WHERE Clause

Filters records based on specified conditions.

4. GROUP BY Clause

Groups rows that have the same values in specified columns, commonly used with aggregate functions.

5. HAVING Clause

Applies conditions to grouped data after aggregation.

6. ORDER BY Clause

Sorts the result set in ascending or descending order.

3. SQL Constraints:

Q.1. What are constraints in SQL? List and explain the different types of constraints.

Ans: Constraints in SQL

Constraints in SQL are rules applied to table columns to ensure the accuracy, consistency, and reliability of data stored in a database. They restrict the type of data that can be inserted into a table and help maintain data integrity.

Constraints are usually defined at the time of table creation or added later using the ALTER command.

Types of Constraints in SQL

1. NOT NULL

Ensures that a column cannot have NULL values.

Purpose:
Prevents empty entries in a column.

2. UNIQUE

Ensures that all values in a column are different.

Purpose:
Avoids duplicate data.

3. PRIMARY KEY

Uniquely identifies each record in a table. It is a combination of UNIQUE and NOT NULL.

Purpose:
Provides a unique identity to every row.

4. FOREIGN KEY

Creates a relationship between two tables by referring to the primary key of another table.

Purpose:
Maintains referential integrity between related tables.

5. CHECK

Ensures that values satisfy a specific condition.

Purpose:
Restricts values based on a rule.

Example: age ≥ 18

6. DEFAULT

Assigns a default value to a column when no value is provided.

Purpose:
Automatically inserts predefined values.

Q.2. How do PRIMARY KEY and FOREIGN KEY constraints differ?

Ans: Difference Between PRIMARY KEY and FOREIGN KEY Constraints

PRIMARY KEY and FOREIGN KEY are SQL constraints used to maintain data integrity and relationships between tables. However, their purposes are different.

PRIMARY KEY

- Uniquely identifies each record in a table.
- Cannot contain NULL values.
- Ensures that no duplicate values exist.
- Only one PRIMARY KEY is allowed per table (though it may contain multiple columns).
- Used to uniquely distinguish every row.

Purpose:
To provide a unique identity to each record in a table.

FOREIGN KEY

- Refers to the PRIMARY KEY of another table.
- Can contain duplicate values.
- Can contain NULL values (unless restricted).
- A table can have multiple FOREIGN KEYS.
- Used to establish relationships between tables.

Purpose:
To maintain referential integrity between related tables.

Q.3. What is the role of NOT NULL and UNIQUE constraints?

Ans: Role of NOT NULL and UNIQUE Constraints in SQL

In SQL, constraints are used to maintain data integrity and accuracy in database tables. Among them, **NOT NULL** and **UNIQUE** are commonly used to restrict the type of values that can be stored in a column.

1. NOT NULL Constraint

The **NOT NULL** constraint ensures that a column cannot contain NULL values.

Role:

- Prevents empty or missing values in a column.
- Ensures that mandatory fields always contain valid data.
- Improves data reliability and completeness.

Example:

If a column like email is defined as NOT NULL, every record must have an email value.

2. UNIQUE Constraint

The **UNIQUE** constraint ensures that all values in a column are different.

Role:

- Prevents duplicate values in a column.
- Maintains data uniqueness.
- Helps identify records uniquely (except NULL values, depending on database system).

Example:

If a column like username is defined as UNIQUE, no two users can have the same username.

4. Main SQL Commands and Sub-commands (DDL):

Q.1. Define the SQL Data Definition Language (DDL).

Ans: SQL Data Definition Language (DDL)

Data Definition Language (DDL) is a subset of SQL used to define, create, modify, and delete the structure of database objects. It controls the schema of a database, including tables, views, indexes, and constraints.

DDL statements are mainly concerned with the design and organization of the database rather than the data stored inside it.

Common DDL Commands

- **CREATE** – Creates database objects such as tables or databases.
- **ALTER** – Modifies existing database objects.
- **DROP** – Deletes database objects permanently.
- **TRUNCATE** – Removes all records from a table while keeping its structure.
- **RENAME** – Changes the name of a database object.

Q.2. Explain the CREATE command and its syntax.

Ans: CREATE Command in SQL

The **CREATE** command is a Data Definition Language (DDL) statement used to create new database objects such as databases, tables, views, and indexes. It defines the structure of these objects by specifying column names, data types, and constraints.

The CREATE command is executed only once for an object unless it is dropped and recreated.

Syntax of CREATE TABLE

```
CREATE TABLE table_name (
column1 datatype constraint,
 column2 datatype constraint,
   ...
);
```

Example

```
CREATE TABLE students (
   id INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   age INT,
   email VARCHAR(100) UNIQUE
);
```

Explanation:

- students is the table name.
- id is defined as PRIMARY KEY.

- name cannot be NULL.
- email must be unique.
- Each column has a datatype and optional constraints.

CREATE DATABASE Syntax

CREATE DATABASE database_name;

Example:

CREATE DATABASE college;

Q.3. What is the purpose of specifying data types and constraints during table creation?

Ans: Purpose of Specifying Data Types and Constraints During Table Creation

When creating a table in SQL, specifying **data types** and **constraints** is essential to define how data is stored and controlled in the database. They help ensure accuracy, consistency, and reliability of information.

Purpose of Data Types

Data types define the kind of values a column can store, such as integers, text, or dates.

Main purposes:

1. **Ensures Correct Data Storage**
   Each column accepts only valid data of the defined type.
2. **Improves Performance**
   Proper data types optimize memory usage and query speed.
3. **Prevents Invalid Data Entry**
   For example, numeric columns cannot store text values.
4. **Defines Data Format**
   Controls how values are stored and displayed.

Purpose of Constraints

Constraints apply rules to table columns to maintain data integrity.

Main purposes:

1. **Maintain Data Accuracy**
   Prevents incorrect or duplicate entries.

2. **Ensure Data Consistency**
   Enforces relationships between tables.
3. **Restrict Invalid Values**
   Using constraints like CHECK and NOT NULL.
4. **Provide Unique Identification**
   Through PRIMARY KEY and UNIQUE constraints.
5. **Protect Referential Integrity**
   With FOREIGN KEY constraints.

5. ALTER Command:

Q.1. What is the use of the ALTER command in SQL?

Ans: Use of the ALTER Command in SQL

The ALTER command is a Data Definition Language (DDL) statement used to modify the structure of an existing database object, such as a table. It allows changes to be made without deleting and recreating the table.

The ALTER command is commonly used when modifications are required after the table has already been created.

Main Uses of ALTER Command

1. Add a New Column

Used to add a new column to an existing table.

```
ALTER TABLE table_name
ADD column_name datatype;
```

2. Modify a Column

Used to change the datatype or size of an existing column.

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

3. Drop a Column

Used to remove a column from a table.

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

4. Add or Remove Constraints

Used to add constraints like PRIMARY KEY, FOREIGN KEY, or UNIQUE.

Example:

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name PRIMARY KEY (column_name);
```

Q.2. How can you add, modify, and drop columns from a table using ALTER?

Ans: Adding, Modifying, and Dropping Columns Using ALTER in SQL

The ALTER command is used to change the structure of an existing table. It allows users to add new columns, modify existing columns, or remove columns without deleting the table.

1. Adding a Column

The ADD keyword is used to insert a new column into an existing table.

Syntax

```
ALTER TABLE table_name
ADD column_name datatype;
```

Example

```
ALTER TABLE students
ADD email VARCHAR(100);
```

This adds a new column named email to the students table.

2. Modifying a Column

The MODIFY (or ALTER COLUMN in some databases) keyword is used to change the datatype or size of an existing column.

Syntax (MySQL / Oracle)

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

Example

```
ALTER TABLE students
MODIFY name VARCHAR(100);
```

This changes the size of the name column.

3. Dropping a Column

The DROP COLUMN keyword is used to remove a column permanently from a table.

Syntax

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Example

```
ALTER TABLE students
DROP COLUMN email;
```

T his removes the email column from the students table.

6. DROP Command:

Q.1. What is the function of the DROP command in SQL?

Ans: **Function of the DROP Command in SQL**

The **DROP** command is a Data Definition Language (DDL) statement used to permanently delete database objects such as tables, databases, views, or indexes.

When the DROP command is executed, both the structure of the object and all its data are removed from the database. This action is irreversible.

Common Uses of DROP Command

1. Drop a Table

```
DROP TABLE table_name;
```

Deletes the table along with all records.

2. Drop a Database

```
DROP DATABASE database_name;
```

Deletes the entire database permanently.

3. Drop Other Objects

```
DROP VIEW view_name;
DROP INDEX index_name;
```

Removes views or indexes.

Important Characteristics

- Removes both data and structure.
- Cannot be rolled back.
- Used carefully because of permanent deletion.
- Frees storage space.

Q.2. Implications of Dropping a Table from a Database

Ans: Dropping a table using the DROP TABLE command permanently removes the table from the database. This action has several important implications that must be considered carefully.

1. Permanent Data Loss

All records stored in the table are deleted permanently. Once dropped, the data cannot be recovered unless a backup exists.

2. Removal of Table Structure

The table definition, including columns, data types, and constraints, is completely removed.

3. Loss of Relationships

Any relationships created through foreign keys with other tables are broken. Dependent tables may also be affected or cause errors.

4. Impact on Applications

Applications or queries that rely on the dropped table will fail, which may lead to system errors.

5. Cannot Be Rolled Back

The DROP operation is not transactional in most database systems, so it cannot be undone.

6. Storage Space Is Released

The space used by the table is freed in the database.

7. Data Manipulation Language (DML):

Q.1. Define the INSERT, UPDATE, and DELETE commands in SQL.

Ans: INSERT, UPDATE, and DELETE Commands in SQL

INSERT, UPDATE, and DELETE are **Data Manipulation Language (DML)** commands in SQL. They are used to add, modify, and remove data in database tables.

1. INSERT Command

The **INSERT** command is used to add new records (rows) into a table.

Syntax

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example

```
INSERT INTO students (id, name, age)
VALUES (1, 'Amit', 20);
```

2. UPDATE Command

The **UPDATE** command is used to modify existing records in a table.

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Example

```
UPDATE students
SET age = 21
WHERE id = 1;
```

3. DELETE Command

The **DELETE** command is used to remove records from a table.

Syntax

```
DELETE FROM table_name
WHERE condition;
```

Example

```
DELETE FROM students
WHERE id = 1;
```

Q.2. What is the importance of the WHERE clause in UPDATE and DELETE operations?

Ans: Importance of the WHERE Clause in UPDATE and DELETE Operations

The **WHERE** clause is used to specify conditions that determine which records are affected by the UPDATE and DELETE commands. It plays a critical role in controlling data modification and preventing accidental changes to the entire table.

Role in UPDATE Operation

In an UPDATE statement, the WHERE clause ensures that **only specific rows are modified**.

Without WHERE, **all records in the table will be updated**.

Example:

```
UPDATE students
SET age = 21
WHERE id = 5;
```

This updates the age only for the student whose id is 5.

If WHERE is omitted:

```
UPDATE students
SET age = 21;
```

All students' ages will be changed.

Role in DELETE Operation

In a DELETE statement, the WHERE clause specifies **which records should be removed**.

Without WHERE, **all rows in the table will be deleted**.

Example:

```
DELETE FROM students
WHERE id = 5;
```

This deletes only the record with id = 5.

If WHERE is omitted:

```
DELETE FROM students;
```

All records in the table are deleted.

Key Importance Points

1. Prevents accidental mass updates or deletions.
2. Ensures precise control over affected records.
3. Protects data integrity.
4. Allows targeted data modification.
5. Reduces risk of data loss.

8. Data Query Language (DQL):

Q.1. What is the SELECT statement, and how is it used to query data?

Ans: SELECT Statement in SQL and Its Use in Querying Data

The **SELECT** statement is one of the most important SQL commands. It is used to retrieve data from one or more tables in a database. The SELECT statement allows users to specify which columns they want to display and apply conditions to filter records.

It forms the foundation of data querying in relational databases.

General Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Explanation of Clauses

SELECT

Specifies the columns to be retrieved.

FROM

Specifies the table from which data is fetched.

WHERE

Filters records based on given conditions (optional).

Example

```
SELECT name, age
FROM students
WHERE age > 18;
```

This query retrieves the name and age of students whose age is greater than 18.

Uses of SELECT Statement

1. Retrieve specific columns or all columns from a table.
2. Filter data using conditions.
3. Sort results using ORDER BY.
4. Group records using GROUP BY.
5. Perform calculations using aggregate functions like SUM, COUNT, and AVG.
6. Combine data from multiple tables using joins.

Q.2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

Ans: Use of the WHERE and ORDER BY Clauses in SQL Queries

The **WHERE** and **ORDER BY** clauses are used in SQL to filter and organize query results. They help control which records are retrieved and how they are displayed.

WHERE Clause

The **WHERE** clause is used to filter records based on specified conditions. It restricts the rows returned by a query.

Purpose of WHERE:

- Selects only required records
- Applies conditions to data
- Prevents unnecessary rows from being fetched
- Helps retrieve precise results

Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example

```
SELECT name, age
FROM students
WHERE age > 18;
```

This query retrieves only students whose age is greater than 18.

ORDER BY Clause

The **ORDER BY** clause is used to sort the result set in ascending or descending order.

By default, sorting is **ascending (ASC)**. Descending order is specified using **DESC**.

Purpose of ORDER BY:

- Arranges data in a readable format
- Sorts records alphabetically or numerically
- Helps in reporting and analysis

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column_name ASC|DESC;
```

Example

```
SELECT name, age
FROM students
ORDER BY age DESC;
```

This query displays students ordered by age from highest to lowest.

Combined Usage Example

```
SELECT name, age
FROM students
WHERE age > 18
ORDER BY age ASC;
```

This retrieves students older than 18 and sorts them by age in ascending order.

9. Data Control Language (DCL):

Q.1. What is the purpose of GRANT and REVOKE in SQL?

Ans: Purpose of GRANT and REVOKE in SQL

GRANT and **REVOKE** are **Data Control Language (DCL)** commands in SQL. They are used to manage user permissions and control access to database objects such as tables, views, and procedures.

These commands help maintain database security by allowing administrators to decide who can perform specific operations.

GRANT Command

The **GRANT** command is used to **give permissions** to users or roles.

Purpose of GRANT:

- Allows users to access database objects
- Assigns privileges such as SELECT, INSERT, UPDATE, DELETE
- Controls who can perform operations on tables
- Supports secure multi-user environments

Syntax

```
GRANT privilege_name
ON object_name
TO user_name;
```

Example

```
GRANT SELECT ON students TO user1;
```

This allows user1 to read data from the students table.

REVOKE Command

The **REVOKE** command is used to **remove previously granted permissions**.

Purpose of REVOKE:

- Withdraws user privileges
- Prevents unauthorized access
- Helps manage changing roles or responsibilities
- Improves data security

Syntax

```
REVOKE privilege_name
ON object_name
FROM user_name;
```

Example

REVOKE SELECT ON students FROM user1;

This removes SELECT permission from user1.

Q.2. How do you manage privileges using these commands?

Ans: Managing Privileges Using GRANT and REVOKE Commands in SQL

Privileges in SQL define what actions a user is allowed to perform on database objects. These privileges are managed using the **GRANT** and **REVOKE** commands. Together, they help control access, maintain security, and support multi-user environments.

1. Granting Privileges

The **GRANT** command is used to assign permissions to users or roles.

Syntax

```
GRANT privilege_name
ON object_name
TO user_name;
```

Example

GRANT SELECT, INSERT ON students TO user1;

This allows user1 to read and insert data into the students table.

2. Granting Privileges with GRANT OPTION

The **GRANT OPTION** allows a user to pass granted privileges to other users.

GRANT SELECT ON students TO user1 WITH GRANT OPTION;

Now user1 can grant SELECT permission to others.

3. Revoking Privileges

The **REVOKE** command is used to remove permissions that were previously granted.

Syntax

```
REVOKE privilege_name
ON object_name
FROM user_name;
```

Example

REVOKE INSERT ON students FROM user1;

This removes INSERT permission from user1.

Key Points in Privilege Management

1. Administrators control who can access database objects.
2. Specific privileges can be granted or revoked as needed.
3. Access can be limited to particular operations (SELECT, UPDATE, DELETE).
4. Privileges can be adjusted when user roles change.
5. Helps prevent unauthorized data access and modification.

10. Transaction Control Language (TCL):

Q.1. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

Ans: Purpose of COMMIT and ROLLBACK Commands in SQL

COMMIT and **ROLLBACK** are **Transaction Control Language (TCL)** commands in SQL. They are used to manage transactions and ensure data consistency in a database.

A transaction is a sequence of SQL operations treated as a single logical unit of work.

COMMIT Command

The **COMMIT** command is used to permanently save all changes made during the current transaction.

Purpose of COMMIT:

- Makes all changes permanent in the database
- Ends the current transaction successfully
- Ensures data is stored permanently
- Releases database locks held by the transaction

Syntax

```
COMMIT;
```

Example

```
UPDATE students SET age = 21 WHERE id = 1;
COMMIT;
```

This permanently saves the update.

ROLLBACK Command

The **ROLLBACK** command is used to undo all changes made during the current transaction before COMMIT is executed.

Purpose of ROLLBACK:

- Cancels uncommitted changes
- Restores the database to its previous state
- Helps recover from errors
- Prevents accidental data modification

Syntax

```
ROLLBACK;
```

Example

```
DELETE FROM students WHERE id = 1;
ROLLBACK;
```

This restores the deleted record (if COMMIT was not issued).

Key Difference

COMMIT	**ROLLBACK**
Saves changes permanently	Undoes uncommitted changes
Ends transaction successfully	Cancels transaction
Cannot be undone	Reverts to previous state

Q.2. Explain how transactions are managed in SQL databases.

Ans: A **transaction** in an SQL database is a group of one or more operations (such as INSERT, UPDATE, DELETE) that are executed together as a single unit of work. Transactions help maintain the accuracy and consistency of data in the database.

If all the operations inside a transaction are successful, the changes are saved permanently. If any operation fails, all changes made in that transaction are undone.

1. ACID Properties

Transactions follow four important properties known as **ACID**:

1. **Atomicity**
   This means “all or nothing.”
   If one part of the transaction fails, the whole transaction fails.
2. **Consistency**
   The database must remain in a valid state before and after the transaction.
3. **Isolation**
   Transactions are executed independently. One transaction does not affect another running transaction.
4. **Durability**
   Once a transaction is committed, the changes are permanently saved, even if the system crashes.

2. Transaction Control Commands

SQL provides special commands to manage transactions:

1. BEGIN (or START TRANSACTION)

It starts a new transaction.

BEGIN;

2. COMMIT

It saves all changes made during the transaction permanently.

COMMIT;

3. ROLLBACK

It cancels all changes made during the transaction.

ROLLBACK;

4. SAVEPOINT

It creates a temporary point inside a transaction.
You can roll back to that specific point without cancelling the entire transaction.

SAVEPOINT sp1;
ROLLBACK TO sp1;

3. Example of a Transaction

Suppose a bank transfers ₹1000 from Account A to Account B.

BEGIN;

UPDATE accounts
SET balance = balance - 1000

```
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE account_id = 2;

COMMIT;
```

If any update fails, we use:

```
ROLLBACK;
```

This ensures that money is not deducted without being added to the other account.

4. Why Transactions Are Important

- They prevent data loss.
- They maintain data integrity.
- They ensure safe handling of multi-step operations.
- They protect data during system failures.

11. SQL Joins:

Q.1. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

Ans: A **JOIN** in SQL is used to combine data from two or more tables based on a related column between them.

In relational databases, data is usually stored in different tables to avoid duplication. To retrieve related data from multiple tables, we use JOIN.

For example, one table may store **customer details**, and another table may store **orders**. Both tables can be connected using a common column like customer_id.

Types of JOIN in SQL

There are mainly four types of JOIN:

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN

1. INNER JOIN

INNER JOIN returns only the matching records from both tables.

If there is no match in either table, that row will not be included in the result.

Example:

```
SELECT customers.name, orders.order_id
    FROM customers
    INNER JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Meaning:

Only customers who have placed orders will appear in the result.

2. LEFT JOIN (LEFT OUTER JOIN)

LEFT JOIN returns all records from the left table and only the matching records from the right table.

If there is no match in the right table, NULL values are returned.

Example:

```
SELECT customers.name, orders.order_id
    FROM customers
    LEFT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Meaning:

All customers will appear in the result.

If a customer has not placed any order, the order columns will show NULL.

3. RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT JOIN returns all records from the right table and only the matching records from the left table.

If there is no match in the left table, NULL values are returned.

Example:

```
SELECT customers.name, orders.order_id
    FROM customers
    RIGHT JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Meaning:

All orders will appear in the result.

If an order does not have a matching customer, customer columns will show NULL.

4. FULL OUTER JOIN

FULL OUTER JOIN returns all records from both tables.

If there is no match, NULL values are shown for the missing side.

Example:

```
SELECT customers.name, orders.order_id
    FROM customers
    FULL OUTER JOIN orders
    ON customers.customer_id = orders.customer_id;
```

Meaning:

- All customers are included
- All orders are included
- If there is no matching data, NULL values are displayed

Simple Difference Between Them

JOIN Type	Returns Matching Rows	Returns Non-Matching Rows
INNER JOIN	Yes	No
LEFT JOIN	Yes	Yes (from left table)
RIGHT JOIN	Yes	Yes (from right table)
FULL OUTER JOIN	Yes	Yes (from both tables)

Q.2. How are joins used to combine data from multiple tables?

Ans: In SQL, **joins** are used to combine data from two or more tables based on a related column between them.

In a relational database, data is usually stored in separate tables to avoid repetition. These tables are connected using a common field, such as an ID. A JOIN helps us bring related data together in one result.

1. Why We Need Joins

Suppose we have two tables:

Customers Table

customer_id	name
1	Amit
2	Rahul
3	Sneha

Orders Table

order_id	customer_id	amount
101	1	5000
102	2	3000
103	1	2000

Here:

- Customer information is stored in one table.
- Order information is stored in another table.
- Both tables are connected using customer_id.

If we want to see **customer name and order amount together**, we must use a JOIN.

2. Basic Syntax of JOIN

```
SELECT column_names
FROM table1
JOIN table2
ON table1.common_column = table2.common_column;
```

The ON condition tells SQL how the tables are related.

3. Example of Combining Data

```
SELECT customers.name, orders.amount
FROM customers
INNER JOIN orders
ON customers.customer_id = orders.customer_id;
```

Result:

name	amount
Amit	5000
Rahul	3000
Amit	2000

This query combines data from both tables using the common column customer_id.

4. Using Multiple Joins

We can also join more than two tables.

Example:
If we have a third table called Payments, we can join all three tables:

```
SELECT customers.name, orders.order_id, payments.payment_date
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id
JOIN payments
ON orders.order_id = payments.order_id;
```

This allows us to collect related information from multiple tables in a single query.

5. How Joins Actually Work

When a JOIN is executed:

1. SQL checks each row in one table.
2. It compares it with rows in the other table.
3. If the condition in the ON clause matches, the rows are combined into one result row.

12. SQL Group By:

Q.1. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

Ans: The **GROUP BY** clause in SQL is used to group rows that have the same values in specified columns.

It is mainly used together with **aggregate functions** like:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

GROUP BY helps us to perform calculations on each group of data instead of the whole table.

Why GROUP BY is Needed

Suppose we have a table called orders.

order_id	customer_id	amount
1	101	5000
2	102	3000
3	101	2000
4	103	4000
5	102	1500

If we want to calculate the **total amount spent by each customer**, we use GROUP BY.

Basic Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Example 1: Using SUM()

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

Result:

customer_id	total_spent
101	7000
102	4500
103	4000

Here:

- Rows are grouped by customer_id.
- SUM(amount) calculates the total for each group.

Example 2: Using COUNT()

To count how many orders each customer placed:

```
SELECT customer_id, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id;
```

Example 3: Using AVG()

To find the average order amount for each customer:

```
SELECT customer_id, AVG(amount) AS average_amount
FROM orders
GROUP BY customer_id;
```

Important Rules

1. Every column in the SELECT statement must either:
    - Be included in the GROUP BY clause, or
    - Be used inside an aggregate function.
2. GROUP BY is usually used with aggregate functions.
3. If GROUP BY is not used, aggregate functions calculate the result for the entire table.

Difference Between With and Without GROUP BY

Without GROUP BY:

```
SELECT SUM(amount) FROM orders;
```

This gives total amount of all orders.

With GROUP BY:

```
SELECT customer_id, SUM(amount)
FROM orders
GROUP BY customer_id;
```

This gives total amount per customer.

Q.2. Explain the difference between GROUP BY and ORDER BY.

Ans: Difference Between GROUP BY and ORDER BY in SQL

Both **GROUP BY** and **ORDER BY** are used in SQL queries, but they have different purposes.

1. GROUP BY

The **GROUP BY** clause is used to group rows that have the same values in a specific column.

It is mainly used with **aggregate functions** like:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

Purpose:

To summarize data by creating groups.

Example:

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id;
```

Here:

- Rows are grouped by customer_id.
- SUM(amount) calculates the total for each customer.

So GROUP BY changes how data is calculated.

2. ORDER BY

The **ORDER BY** clause is used to sort the result in ascending (ASC) or descending (DESC) order.

Purpose:

To arrange the result in a specific order.

Example:

```
SELECT customer_id, amount
FROM orders
ORDER BY amount DESC;
```

Here:

- The data is sorted from highest amount to lowest.
- It does not group or summarize data.
- It only changes the order of display.

Main Differences

GROUP BY	**ORDER BY**
Groups rows with similar values	Sorts rows in a specific order
Used with aggregate functions	Used for arranging results
Reduces number of rows (by grouping)	Does not reduce rows
Affects calculation of results	Affects only display order

Using Both Together

We can use both in the same query.

Example:

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id
ORDER BY total_amount DESC;
```

- GROUP BY creates groups for each customer.
- SUM calculates total amount per customer.
- ORDER BY sorts the customers by total amount in descending order.

13. SQL Stored Procedure:

Q.1. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

Ans: A **stored procedure** is a set of SQL statements that is saved in the database and can be executed whenever needed.

It works like a small program inside the database. Once created, it can be called many times without rewriting the SQL code again.

Stored procedures can contain:

- Multiple SQL statements
- Conditions (IF, ELSE)
- Loops
- Variables
- Input and output parameters

Example of a Stored Procedure

```
CREATE PROCEDURE GetCustomerOrders
AS
BEGIN
  SELECT * FROM orders;
END;
```

To execute it:

```
EXEC GetCustomerOrders;
```

This procedure will run the saved SQL query.

Stored Procedure with Parameter

```
CREATE PROCEDURE GetOrdersByCustomer
  @CustomerID INT
AS
BEGIN
  SELECT * FROM orders
  WHERE customer_id = @CustomerID;
END;
```

To execute:

```
EXEC GetOrdersByCustomer 101;
```

Here, @CustomerID is a parameter.

What is a Standard SQL Query?

A **standard SQL query** is a single SQL statement that is written and executed directly.

Example:

SELECT * FROM orders WHERE customer_id = 101;

This query runs once and is not saved permanently in the database.

Difference Between Stored Procedure and Standard SQL Query

Stored Procedure	**Standard SQL Query**
Saved inside the database	Not saved permanently
Can contain multiple SQL statements	Usually a single statement
Can accept parameters	Parameters must be written each time
Can include logic (loops, conditions)	Limited to simple operations
Reusable	Must be rewritten each time
More secure (can restrict direct table access)	Less secure if users access tables directly

Advantages of Stored Procedures

1. Reusability – Write once, use many times.
2. Better performance – Compiled and optimized by the database.
3. Security – Users can execute the procedure without direct access to tables.
4. Easy maintenance – Changes can be made in one place.

Q.2. Explain the advantages of using stored procedures.

Ans: A **stored procedure** is a group of SQL statements that is saved in the database and can be executed whenever needed. Using stored procedures provides several important advantages.

1. Reusability

Stored procedures are written once and can be used many times.

Instead of writing the same SQL query again and again in different programs, we can simply call the stored procedure. This saves time and reduces repetition.

2. Better Performance

Stored procedures are stored and compiled inside the database.

When they are executed:

- The database does not need to recompile the SQL code every time.
- Execution is faster compared to running separate SQL statements.

This improves overall system performance.

3. Improved Security

Stored procedures increase security because:

- Users can be given permission to execute the procedure.
- They do not need direct access to the actual tables.

This helps protect sensitive data and prevents unauthorized access.

4. Reduced Network Traffic

When multiple SQL statements are executed separately, each statement is sent to the database server.

With a stored procedure:

- All SQL statements are executed together on the server.
- Only the procedure call is sent from the application.

This reduces network communication and improves efficiency.

5. Easier Maintenance

If business logic changes, we only need to update the stored procedure.

There is no need to change the code in multiple applications.
This makes maintenance simple and organized.

6. Support for Complex Logic

Stored procedures can include:

- IF–ELSE conditions

- Loops
- Variables
- Error handling

This allows complex operations to be handled directly inside the database.

7. Consistency

Since the same stored procedure is used everywhere, it ensures that:

- The same rules are followed.
- The same calculations are performed.
- The same results are produced.

This keeps the system consistent.

14. SQL View:

Q.1. What is a view in SQL, and how is it different from a table?

Ans: A **view** in SQL is a virtual table based on the result of a SQL query.

It does not store data physically like a table. Instead, it shows data from one or more tables whenever it is accessed.

A view is created using a SELECT statement.

Example of Creating a View

```
CREATE VIEW CustomerOrders AS
SELECT customers.name, orders.amount
FROM customers
JOIN orders
ON customers.customer_id = orders.customer_id;
```

Now, instead of writing the full JOIN query every time, we can use:

```
SELECT * FROM CustomerOrders;
```

The view will display the result of the saved query.

What is a Table?

A **table** is a database object that stores data physically in rows and columns.

Example:

```
CREATE TABLE customers (
  customer_id INT,
  name VARCHAR(50),
  city VARCHAR(50)
);
```

Data inserted into a table is permanently stored in the database.

Difference Between View and Table

View	**Table**
Virtual table	Physical table
Does not store data	Stores data permanently
Based on a SQL query	Stores actual rows and columns
Takes less storage	Takes storage space
Used for security and simplification	Used to store and manage data

Advantages of Views

1. **Security**
   Users can access a view without seeing the actual tables.
2. **Simplification**
   Complex queries can be saved as views and reused easily.
3. **Data Abstraction**
   Users do not need to know how tables are structured.

Q.2. Explain the advantages of using viewsin SQL databases.

Ans: A **view** is a virtual table created from a SQL query. It does not store data physically but shows data from one or more tables. Using views provides many advantages in database systems.

1. Improved Security

Views help protect sensitive data.

- Users can be given access to a view instead of the actual table.

- Sensitive columns (like salary or password) can be hidden.
- Users can only see the data allowed in the view.

This improves data security and privacy.

2. Simplifies Complex Queries

If a query involves multiple joins and conditions, it can become long and difficult to understand.

A view allows you to:

- Save the complex query once.
- Use it easily whenever needed.

Example:

Instead of writing a long JOIN query every time, you can simply use:

SELECT * FROM CustomerOrders;

This makes the system easier to use.

3. Data Abstraction

Views hide the internal structure of the database.

Users:

- Do not need to know how many tables are involved.
- Do not need to understand complex relationships.

They only see the final result provided by the view.

4. Consistency

Since the same view is used by different users or applications:

- The same logic is applied everywhere.
- The same calculations are performed.
- Results remain consistent.

5. Easier Maintenance

If business logic changes, you only need to modify the view.

You do not need to change queries in multiple applications.
This saves time and reduces errors.

6. Restricting Data Access

Views can limit:

- Specific rows (using WHERE condition)
- Specific columns

For example, a view can show only employees from a particular department.

15. SQL Triggers:

Q.1. What is a trigger in SQL? Describe its types and when they are used

Ans: A **trigger** in SQL is a special type of stored program that automatically runs when a specific event happens in a table.

Triggers are executed automatically when we perform operations like:

- INSERT
- UPDATE
- DELETE

They are mainly used to maintain data integrity, enforce rules, and automatically perform actions when data changes.

Basic Example of a Trigger

```
CREATE TRIGGER CheckSalary
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
  IF NEW.salary < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary cannot be negative';
  END IF;
END;
```

This trigger prevents inserting a negative salary.

Types of Triggers

Triggers are mainly classified based on **timing** and **event**.

1. Based on Timing

(a) BEFORE Trigger

- Executes before the actual operation (INSERT, UPDATE, DELETE).
- Used to validate or modify data before saving.

Example use:

- Check if data is valid.
- Prevent wrong data entry.

(b) AFTER Trigger

- Executes after the operation is completed.
- Used when we need to perform an action after data is saved.

Example use:

- Record changes in an audit table.
- Send notification after inserting a record.

2. Based on Event

(a) INSERT Trigger

Runs automatically when a new record is inserted.

Used for:

- Validation
- Logging new records

(b) UPDATE Trigger

Runs when an existing record is updated.

Used for:

- Tracking changes
- Maintaining history of data

(c) DELETE Trigger

Runs when a record is deleted.

Used for:

- Preventing accidental deletion
- Storing deleted data in backup table

Example of AFTER INSERT Trigger

```
CREATE TRIGGER LogNewOrder
AFTER INSERT
ON orders
FOR EACH ROW
BEGIN
  INSERT INTO order_log(order_id, log_date)
  VALUES (NEW.order_id, NOW());
END;
```

This trigger automatically logs new orders.

When Are Triggers Used?

Triggers are commonly used for:

1. Enforcing business rules
2. Validating data
3. Maintaining audit logs
4. Automatically updating related tables
5. Preventing invalid operations

Q.2. Explain the difference between INSERT, UPDATE, and DELETE triggers.

Ans: A **trigger** is a special SQL program that automatically runs when a specific event happens in a table.

There are three main types of triggers based on the event that causes them to run:

- INSERT trigger
- UPDATE trigger
- DELETE trigger

Each one works when a different type of operation is performed.

1. INSERT Trigger

An **INSERT trigger** runs automatically when a new record is added to a table.

When it is used:

- To validate new data before saving
- To record new entries in a log table
- To automatically update related tables

Example situation:

If a new order is inserted, we may want to:

- Record it in an audit table
- Update stock quantity

2. UPDATE Trigger

An **UPDATE trigger** runs when an existing record is modified.

When it is used:

- To track changes in important data
- To store old values in a history table
- To prevent incorrect updates

Example situation:

If an employee's salary is updated, we may:

- Save the old salary in a salary_history table
- Check that the new salary is not negative

3. DELETE Trigger

A **DELETE trigger** runs when a record is removed from a table.

When it is used:

- To prevent accidental deletion
- To store deleted records in a backup table
- To update related records

Example situation:

If a customer record is deleted, we may:

- Save the deleted data in an archive table
- Remove related records from another table

Main Differences

Trigger Type	Executes When	Common Use
INSERT Trigger	A new record is added	Validate or log new data
UPDATE Trigger	A record is modified	Track or control changes
DELETE Trigger	A record is removed	Backup or prevent deletion

16. Introduction to PL/SQL:

Q.1. What is PL/SQL, and how does it extend SQL's capabilities?

Ans: **PL/SQL** stands for *Procedural Language/Structured Query Language*.

It is a programming language developed by Oracle that extends SQL by adding programming features like conditions, loops, and variables.

SQL is mainly used to perform operations such as SELECT, INSERT, UPDATE, and DELETE. PL/SQL allows us to write complete programs that include SQL statements along with procedural logic.

PL/SQL is mainly used in **Oracle Database**.

Why PL/SQL is Needed

SQL works well for simple queries, but it has limitations:

- It cannot use loops easily.
- It cannot store complex logic directly.
- It does not support variables like programming languages.

PL/SQL solves these problems by combining SQL with programming concepts.

How PL/SQL Extends SQL Capabilities

PL/SQL adds several powerful features to SQL:

1. Variables

PL/SQL allows declaring and using variables.

Example:

```
DECLARE
  total NUMBER;
BEGIN
  SELECT SUM(amount) INTO total FROM orders;
  DBMS_OUTPUT.PUT_LINE(total);
END;
```

SQL alone cannot store query results in variables like this.

2. Conditional Statements

PL/SQL supports IF–THEN–ELSE conditions.

Example:

```
IF salary > 50000 THEN
  DBMS_OUTPUT.PUT_LINE('High Salary');
ELSE
  DBMS_OUTPUT.PUT_LINE('Low Salary');
END IF;
```

This allows decision-making inside the database.

3. Loops

PL/SQL supports loops such as:

- FOR loop
- WHILE loop
- LOOP

This allows repeated execution of code.

4. Error Handling

PL/SQL provides exception handling using:

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No Data Found');
```

This makes programs more secure and stable.

5. Stored Procedures and Functions

PL/SQL is used to create:

- Stored procedures
- Functions
- Triggers
- Packages

These help in building complex business logic inside the database.

Difference Between SQL and PL/SQL

SQL	**PL/SQL**
Query language	Programming language
Used to manipulate data	Used to write complete programs
Executes one statement at a time	Executes multiple statements together
No support for loops or conditions	Supports loops, conditions, variables

Q.2. List and explain the benefits of using PL/SQL.

Ans: Benefits of Using PL/SQL

PL/SQL is an extension of SQL used mainly in Oracle databases. It combines SQL with programming features such as variables, loops, and conditions. Using PL/SQL provides many important benefits.

1. Combines SQL with Programming Logic

PL/SQL allows you to use SQL statements along with programming features like:

- Variables
- IF–ELSE conditions
- Loops
- Exception handling

This makes it possible to write complete programs inside the database.

2. Improved Performance

PL/SQL blocks are sent to the database server as a single unit.

Instead of sending multiple SQL statements one by one:

- The entire block executes at once.
- This reduces communication between application and database.
- It improves performance.

3. Better Error Handling

PL/SQL provides exception handling using the EXCEPTION block.

This helps:

- Detect errors
- Handle them properly
- Prevent system crashes

Example situations:

- No data found
- Division by zero
- Duplicate values

4. Code Reusability

PL/SQL is used to create:

- Stored procedures
- Functions
- Packages
- Triggers

These can be saved in the database and reused whenever needed. This reduces code duplication.

5. Enhanced Security

Users can be given permission to execute PL/SQL programs without direct access to tables.

This:

- Protects sensitive data
- Restricts unauthorized access
- Improves database security

6. Supports Modular Programming

PL/SQL allows dividing large programs into smaller parts using procedures and functions.

This makes:

- Code easier to manage
- Debugging simpler
- Maintenance easier

7. Maintains Data Integrity

Business rules can be written directly inside PL/SQL programs.

This ensures:

- Data validation
- Correct calculations
- Consistent results

8. Reduces Network Traffic

Since multiple SQL statements are executed in one PL/SQL block:

- Fewer calls are made to the database.
- Network traffic is reduced.

17. PL/SQL Control Structures:

Q.1. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

Ans: Control structures in PL/SQL are used to control the flow of execution of a program.

They help decide:

- Which statements should run
- How many times a statement should run
- What action to take based on a condition

Control structures make PL/SQL programs logical and flexible.

There are mainly three types of control structures:

1. Conditional control (IF–THEN)
2. Loop control (LOOP, WHILE, FOR)
3. Sequential control

In this answer, we will explain **IF–THEN** and **LOOP**.

1. IF–THEN Control Structure

The **IF–THEN** statement is used to execute code only when a condition is true.

Basic Syntax

```
IF condition THEN
  statements;
END IF;
```

If the condition is true, the statements inside the block will execute.
If the condition is false, the statements will not execute.

Example

```
DECLARE
  salary NUMBER := 60000;
BEGIN
  IF salary > 50000 THEN
    DBMS_OUTPUT.PUT_LINE('High Salary');
  END IF;
END;
```

In this example:

- If salary is greater than 50000, it prints "High Salary".

IF–THEN–ELSE Example

```
DECLARE
  marks NUMBER := 45;
BEGIN
  IF marks >= 50 THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Fail');
```

```
  END IF;
END;
```

Here:

- If marks are 50 or more → Pass
- Otherwise → Fail

2. LOOP Control Structure

A **LOOP** is used to repeat a set of statements multiple times.

PL/SQL supports different types of loops, but here we explain the basic LOOP.

Basic LOOP Syntax

```
LOOP
  statements;
  EXIT WHEN condition;
END LOOP;
```

The loop continues until the EXIT condition becomes true.

Example

```
DECLARE
  counter NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(counter);
    counter := counter + 1;
    EXIT WHEN counter > 5;
  END LOOP;
END;
```

This will print numbers from 1 to 5.

Difference Between IF–THEN and LOOP

IF–THEN	**LOOP**
Used for decision making	Used for repetition
Executes once if condition is true	Executes multiple times

IF–THEN	LOOP
Checks condition to choose action	Repeats statements until condition is met

Q.2. How do control structures in PL/SQL help in writing complex queries?

Ans: Control structures in PL/SQL help in controlling how and when SQL statements are executed. They make it possible to add logic, conditions, and repetition to database operations.

In normal SQL, we can write queries to retrieve or modify data. But SQL alone cannot handle complex decision-making or repeated operations easily. PL/SQL control structures solve this problem.

1. Decision Making Using IF–THEN

Control structures like **IF–THEN** allow the program to make decisions.

For example:

- If a customer's balance is low, apply a penalty.
- If marks are above 50, assign grade "Pass".
- If stock is less than minimum level, show warning.

Example:

```
IF total_amount > 10000 THEN
  discount := total_amount * 0.10;
ELSE
  discount := 0;
END IF;
```

This type of logic cannot be written directly using simple SQL queries.

2. Repeating Operations Using LOOPS

Loops help execute SQL statements multiple times.

For example:

- Process each record one by one.
- Update multiple rows based on certain conditions.
- Insert data in bulk with specific calculations.

Example:

```
FOR i IN 1..5 LOOP
  INSERT INTO test_table VALUES (i);
END LOOP;
```

Loops make complex operations easier and more organized.

3. Handling Errors Using EXCEPTION

PL/SQL allows handling errors using exception blocks.

This helps:

- Prevent program crashes
- Handle unexpected problems
- Maintain data integrity

Example:

- If no data is found, show a message.
- If division by zero happens, handle it safely.

4. Combining Multiple SQL Statements

Control structures allow multiple SQL queries to run inside one PL/SQL block.

For example:

- First check a condition.
- Then update a table.
- Then insert data into another table.
- If something fails, handle the error.

This makes complex business logic possible inside the database.

5. Better Organization of Logic

Control structures make programs:

- More readable
- Easier to maintain
- Structured and organized

Instead of writing long and confusing queries, we can divide logic into smaller steps.

18. SQL Cursors:

Q.1. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

Ans: A **cursor** in PL/SQL is a pointer that is used to retrieve and process multiple rows returned by a SQL query.

When a SELECT statement returns more than one row, PL/SQL needs a way to handle each row one by one. A cursor helps in fetching those rows individually.

In simple words, a cursor allows PL/SQL to process query results row by row.

Types of Cursors in PL/SQL

There are two types of cursors:

1. Implicit Cursor
2. Explicit Cursor

1. Implicit Cursor

An **implicit cursor** is automatically created by Oracle whenever a SQL statement is executed.

It is used for:

- INSERT
- UPDATE
- DELETE
- SELECT statements that return only one row

The programmer does not need to declare or open it.

Example:

```
BEGIN
  UPDATE employees
  SET salary = salary + 1000
  WHERE department_id = 10;
END;
```

Here:

- Oracle automatically creates an implicit cursor.
- We do not manage it manually.

Oracle also provides attributes like:

- SQL%ROWCOUNT
- SQL%FOUND
- SQL%NOTFOUND

These help check the result of the operation.

2. Explicit Cursor

An **explicit cursor** is created by the programmer when a SELECT statement returns multiple rows.

The programmer must:

1. Declare the cursor
2. Open the cursor
3. Fetch rows
4. Close the cursor

Example:

```
DECLARE
  CURSOR emp_cursor IS
    SELECT name, salary FROM employees;

  emp_name employees.name%TYPE;
  emp_salary employees.salary%TYPE;
BEGIN
  OPEN emp_cursor;
  LOOP
    FETCH emp_cursor INTO emp_name, emp_salary;
    EXIT WHEN emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(emp_name || ' ' || emp_salary);
  END LOOP;
  CLOSE emp_cursor;
END;
```

Here:

- We manually control the cursor.
- Each row is processed one by one.

Difference Between Implicit and Explicit Cursors

Implicit Cursor	Explicit Cursor
Created automatically by Oracle	Created manually by programmer
Used for single-row queries and DML statements	Used for multi-row SELECT queries
No need to open or close	Must be opened, fetched, and closed
Simple and automatic	More control and flexible

Q.2. When would you use an explicit cursor over an implicit one?

Ans: In PL/SQL, an **implicit cursor** is automatically created by Oracle for simple SQL statements. It works well when:

- The query returns only one row
- You are using INSERT, UPDATE, or DELETE
- You do not need to process rows one by one

However, we use an **explicit cursor** when you need more control over multiple rows returned by a SELECT statement.

Situations Where Explicit Cursor Is Used

1. When the Query Returns Multiple Rows

If a SELECT statement returns many rows and you need to process each row separately, you should use an explicit cursor.

Example situation:

- Display each employee's details one by one
- Perform calculations for every row
- Update another table based on each record

Implicit cursors cannot handle multiple rows in this way.

2. When You Need Row-by-Row Processing

If business logic must be applied individually to each record, explicit cursors are useful.

For example:

- Check each employee's salary and give a bonus only if eligible
- Send notifications for each order

This requires fetching and handling one row at a time.

3. When You Need More Control

Explicit cursors allow you to:

- Open and close the cursor manually
- Control when data is fetched
- Use cursor attributes like %FOUND and %NOTFOUND
- Pause or stop processing when needed

This level of control is not available in implicit cursors.

4. When Complex Logic Is Required

If the program needs:

- Loops combined with conditions
- Custom error handling for each row
- Step-by-step processing

Explicit cursors are more suitable.

When Not to Use Explicit Cursor

You should not use explicit cursors when:

- The query returns only one row
- You are performing simple INSERT, UPDATE, DELETE
- No row-by-row logic is required

In such cases, implicit cursors are simpler and better.

19. Rollback and Commit Savepoint:

Q.1. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

Ans: A **SAVEPOINT** is a point created inside a transaction that allows you to roll back part of the transaction instead of canceling the entire transaction.

SAVEPOINT acts like a checkpoint inside a transaction.

Normally:

- If we use ROLLBACK, all changes in the transaction are undone.
- But with SAVEPOINT, we can undo changes only up to a specific point.

Why SAVEPOINT Is Needed

Sometimes a transaction contains multiple steps.
If one step fails, we may not want to cancel everything.

SAVEPOINT helps us:

- Undo only specific operations
- Keep earlier successful operations
- Handle errors more safely

Syntax of SAVEPOINT

```
SAVEPOINT savepoint_name;
```

Example of SAVEPOINT

```
BEGIN;

    INSERT INTO orders VALUES (1, 'Amit', 5000);

    SAVEPOINT sp1;

    INSERT INTO orders VALUES (2, 'Rahul', 3000);

    ROLLBACK TO sp1;

    COMMIT;
```

What Happens Here:

1. First INSERT is executed.
2. SAVEPOINT sp1 is created.
3. Second INSERT is executed.
4. ROLLBACK TO sp1 removes only the second INSERT.
5. COMMIT saves the first INSERT permanently.

So only the first record remains.

How ROLLBACK Works with SAVEPOINT

There are two types of rollback:

1. ROLLBACK (without savepoint)

ROLLBACK;

- Cancels the entire transaction.
- All changes are undone.
- All savepoints are removed.

2. ROLLBACK TO savepoint_name

ROLLBACK TO sp1;

- Undoes changes made after that savepoint.
- Changes before the savepoint remain.
- The savepoint still exists (unless the whole transaction is rolled back).

How COMMIT Works with SAVEPOINT

COMMIT;

- Saves all changes permanently.
- Removes all savepoints.
- Ends the transaction.

After COMMIT:

- we cannot roll back to a savepoint.
- The transaction is complete.

Summary of Interaction

Command	**Effect on Savepoints**
SAVEPOINT	Creates a checkpoint inside transaction
ROLLBACK	Cancels entire transaction and removes savepoints
ROLLBACK TO savepoint	Cancels changes after savepoint

Command	Effect on Savepoints
COMMIT	Saves changes and removes all savepoints

Q.2. When is it useful to use savepoints in a database transaction?

Ans: A **SAVEPOINT** is useful when a transaction has multiple steps and you want the option to undo only part of the transaction instead of cancelling everything.

It acts like a checkpoint inside a transaction.

1. When a Transaction Has Multiple Steps

If a transaction performs many operations such as:

- Insert data
- Update data
- Delete data

and one step fails, you may not want to undo all previous successful steps.

Using SAVEPOINT allows you to roll back only the part that failed.

2. When Handling Complex Business Logic

In real applications, a single transaction may include:

- Checking conditions
- Updating multiple tables
- Logging information

If one part of the logic fails, you can roll back to a specific savepoint instead of cancelling the whole process.

This is useful in systems like:

- Banking transactions
- E-commerce orders
- Inventory management

3. When Testing or Validating Data

Sometimes, after performing some operations, we may want to:

- Check if data is valid
- Perform additional verification

If validation fails, we can roll back to a savepoint without losing earlier correct operations.

4. When Processing Large Data Operations

If a transaction processes many records:

- we can create savepoints at different stages.
- If an error occurs later, we can return to the last safe stage.

This helps in reducing rework.

5. When You Want Partial Rollback

Normally, ROLLBACK cancels the entire transaction.

But if we only want to undo recent changes and keep earlier ones, SAVEPOINT is very useful.

Example situation:

- Insert customer record
- Create savepoint
- Insert order record
- If order fails, roll back only the order, not the customer